

Yudillo 백엔드 — 애노테이션 · 문법 레퍼런스

작성일: 2026-09-28 대상: `C:\Users\PC\Desktop\참고자료\Study\back-end-develop`

이 프로젝트에서 실제로 쓰고 있는 것만 정리했다. 일반적인 Spring 강좌가 아니라, 코드를 읽다가 "이 애노테이션 뭐지?" 할 때 찾아보는 용도다.

소스에서 추출한 결과 애노테이션 **54종**, 자바 문법 요소 **12가지**를 쓰고 있다.

0. 빠른 찾아보기

궁금한 것	장
<code>@Getter</code> 같은 Lombok	1장
<code>@Service</code> , <code>@Bean</code> 같은 스프링 코어	2장
<code>@RestController</code> , <code>@PostMapping</code> 같은 웹	3장
<code>@Entity</code> , <code>@Column</code> 같은 JPA	4장
<code>@NotBlank</code> , <code>@Pattern</code> 같은 검증	5장
<code>@Schema</code> , <code>@Operation</code> 같은 Swagger	6장
<code>@AuthenticationPrincipal</code> 같은 보안	7장
<code>@JsonInclude</code> 같은 Jackson	8장
우리가 직접 만든 애노테이션	9장
<code>record</code> , <code>enum</code> , 람다 같은 자바 문법	10장
헛갈리기 쉬운 것들	11장

1. Lombok — 보일러플레이트 생성

Lombok 은 컴파일 시점에 코드를 생성하고 사라진다. 빌드된 jar 에는 들어가지 않는다. 그래서 `build.gradle` 에서 `compileOnly` + `annotationProcessor` 두 줄이 필요하다.

```
compileOnly 'org.projectlombok:lombok'
annotationProcessor 'org.projectlombok:lombok'
```

@Getter — 7곳

필드의 getter 를 만든다. `boolean` 은 `isXxx()`, 나머지는 `getXxx()`.

```
@Getter
public class AuthUser implements UserDetails {
    private final UUID id;          // → getId()
    private final String email;      // → getEmail()
}
```

`@Setter` 는 이 프로젝트에 하나도 없다. 의도적이다. setter 를 열면 어디서 값이 바뀌었는지 추적이 안 된다. 대신 `changeName()`, `changePassword()` 처럼 의미 있는 메서드만 만든다.

@RequiredArgsConstructor — 11곳

`final` 필드를 받는 생성자를 만든다. 이 프로젝트의 의존성 주입 방식이다.

```
@Service
@RequiredArgsConstructor
public class AuthService {
    private final UserRepository userRepository;
    private final PasswordEncoder passwordEncoder;
    // Lombok 이 AuthService(UserRepository, PasswordEncoder) 생성자를 만든다
    // 스프링이 생성자가 하나뿐이면 @Autowired 없이도 주입한다
}
```

필드 주입(`@Autowired private X x;`) 대신 쓰는 이유

1. `final` 이라 불변이다
2. 의존성이 없으면 객체 생성 자체가 불가능 → 기동 시점에 실패 (필드 주입은 런타임 NPE)
3. 테스트에서 `new AuthService(mockRepo, mockEncoder)` 로 직접 만들 수 있다
4. 생성자 파라미터가 많아지면 "이 클래스가 일을 너무 많이 한다"가 눈에 보인다

@NoArgsConstructor(access = AccessLevel.PROTECTED) — 2곳 (엔티티)

```
@Entity
@NoArgsConstructor(access = AccessLevel.PROTECTED)
public class User extends BaseEntity { ... }
```

JPA 는 리플렉션으로 엔티티를 만들기 때문에 기본 생성자가 반드시 필요하다. 하지만 `public` 으로 열면 아무 데서나 `new User()` 로 빈 객체를 만들 수 있다.

`PROTECTED` 로 두면 JPA 는 쓸 수 있지만 다른 패키지 코드는 못 쓴다. 딱 필요한 만큼만 연다.

@Builder — 1곳

```
@Builder
private User(String email, String password, String name, Role role, boolean emailVerified) {
    this.role = role == null ? Role.USER : role;    // 기본값 처리
    ...
}

// 사용
User user = User.builder()
    .email(request.email())
    .password(passwordEncoder.encode(request.password()))
    .name(request.name())
    .build();
```

생성자 대신 빌더를 쓰는 이유: 파라미터가 5개인데 `new User(email, password, name, ...)` 에서 `email` 과 `name` 순서를 바꿔도 둘 다 **String** 이라 컴파일이 된다. 런타임에 이메일 칸에 이름이 들어가는 버그가 난다. 빌더는 이름으로 지정하니 불가능하다.

생성자가 `private` 이라 빌더 외에는 만들 방법이 없다.

@Slf4j — 5곳

`private static final Logger log = LoggerFactory.getLogger(...)` 를 만든다.

```
@Slf4j
public class GlobalExceptionHandler {
    log.debug("business exception: {} - {}", errorCode.getCode(), e.getMessage());
    log.error("unhandled exception", e);
}
```

`{}` 는 SLF4J 의 자리 표시자다. **문자열 연결(+)**보다 낫다 — 해당 로그 레벨이 꺼져 있으면 문자열을 아예 만들지 않는다.

현재 `AuthService` 에 `@Slf4j` 가 붙어 있지만 **로그를 한 줄도 찍지 않는다**. 로그인·비밀번호 변경 같은 보안 이벤트는 남기는 게 좋다. (코드 리뷰 3.2-E)

2. 스프링 코어 — 빈 등록과 설정

빈 등록 3형제 — @Service / @Component / @Configuration

셋 다 "스프링이 관리하는 객체(빈)로 등록하라"는 뜻이다. 기능은 거의 같고 의도가 다르다.

애노테이션	쓰는 곳	이 프로젝트
<code>@Service</code>	업무 로직	<code>AuthService</code> , <code>UserService</code> , <code>AuthUserDetailsService</code>
<code>@Component</code>	그 외 일반 빈	<code>JwtTokenProvider</code> , <code>JwtAuthenticationFilter</code> , 핸들러 2개
<code>@Configuration</code>	설정 클래스 (<code>@Bean</code> 을 담는 곳)	<code>SecurityConfig</code> , <code>SwaggerConfig</code> 등 5개
<code>@Repository</code>	데이터 접근	안 씀 — Spring Data JPA 가 자동 등록

`UserRepository` 에 `@Repository` 가 없는 이유는, 인터페이스만 선언하면 Spring Data 가 프록시 구현체를 만들어 등록하기 때문이다.

`@Bean` — 9곳

메서드가 반환하는 객체를 빈으로 등록한다. 내가 만든 클래스가 아니어서 `@Component` 를 붙일 수 없을 때 쓴다.

```
@Configuration
public class SecurityConfig {
    @Bean
    public PasswordEncoder passwordEncoder() {
        return new BCryptPasswordEncoder(); // 남의 라이브러리 클래스
    }
}
```

빈 이름은 메서드 이름이다. 이 프로젝트에서 실제로 사고가 났던 지점이다.

```
@Configuration
public class ErrorResponseCustomizer { // ← 이것도 빈 (이름: errorResponseCustomizer)
    @Bean
    public OperationCustomizer errorResponseCustomizer() { ... } // ← 같은 이름 → 기동 실패
}
```

```
The bean 'errorResponseCustomizer' ... already been defined
```

`@Configuration` 클래스 자체도 빈이라 이름이 겹친다. 메서드명을 `errorResponseOperationCustomizer` 로 바꿔 해결했다.

`@ConfigurationProperties` — 3곳

yaml 값을 타입 있는 객체로 받는다.

```
@ConfigurationProperties(prefix = "jwt")
public record JwtProperties(
    @NotBlank String secret,
    String issuer,
    Duration accessTokenValidity,    // "30m" → Duration 자동 변환
    Duration refreshTokenValidity
) { }
```

```
jwt:
  secret: ${JWT_SECRET}
  access-token-validity: 30m      # 케밥 케이스 → 카멜 케이스 자동 매핑
```

`@Value("${jwt.secret}")` 대신 쓰는 이유

1. 관련 설정이 한 클래스에 모인다
2. 타입 변환(`Duration`)과 검증(`@NotBlank`)이 된다
3. IDE 자동완성이 된다

이 프로젝트에서는 `JwtProperties`, `CorsProperties`, `DateTimeProperties` 세 개가 쓴다.

`@ConfigurationPropertiesScan` — 1곳

`ApiApplication` 에 붙어 있다. 이게 없으면 위 3개가 빈으로 등록되지 않아 기동이 실패한다.

```
@EnableJpaAuditing
@ConfigurationPropertiesScan
@SpringBootApplication
public class ApiApplication { ... }
```

대안으로 각 properties 클래스에 `@Component` 를 붙이거나

`@EnableConfigurationProperties(JwtProperties.class)` 를 쓸 수도 있다. 스캔 방식이 클래스가 늘어나도 손덜 게 없어 편하다.

`@Validated` — 1곳

`@ConfigurationProperties` 와 함께 쓰면 **기동 시점에 설정값을 검증**한다.

```
@Validated
@ConfigurationProperties(prefix = "jwt")
public record JwtProperties(@NotBlank String secret, ...) { }
```

`JWT_SECRET` 이 비어 있으면 애플리케이션이 뜨지 않는다. 런타임에 토큰 발급이 실패하는 것보다 낫다.

`@Transactional` — 9곳

메서드를 하나의 DB 트랜잭션으로 묶는다. 스프링이 **프록시**로 감싼다.

```
// 우리가 쓴 것
@Transactional
public UserResponse signup(SignupRequest request) { ... }

// 스프링이 실제로 하는 일 (개념)
try {
    트랜잭션_시작();
    결과 = 원래메서드();
    커밋();
} catch (RuntimeException e) {
    롤백();
    throw e;
}
```

알아야 할 3가지

① 기본적으로 `RuntimeException` 에만 롤백한다. 체크 예외는 롤백하지 않는다. `BusinessException` 이 `RuntimeException` 을 상속한 이유가 이것이다.

② `readOnly = true` 는 최적화다.

```
@Transactional(readOnly = true)
public UserResponse getMe(UUID userId) { ... }
```

JPA 가 변경 감지용 스냅샷을 만들지 않아 메모리와 CPU 를 아낀다. 읽기 전용 복제본으로 라우팅할 때 기준이 되기도 한다.

③ 같은 클래스 안에서 호출하면 동작하지 않는다. 가장 많이 당하는 함정이다.

```
@Service
public class SomeService {
    @Transactional
    public void a() {
        b(); // ← 프록시를 안 거쳐서 b() 의 @Transactional 이 무시됨
    }

    @Transactional(propagation = REQUIRES_NEW)
    public void b() { ... }
}
```

전파 옵션을 다르게 하려면 클래스를 분리해야 한다.

@EnableJpaAuditing — 1곳

`BaseTimeEntity` 의 `@CreatedDate` , `@LastModifiedDate` 를 동작하게 한다.

빠뜨리면 값이 null 로 들어가고 `nullable = false` 제약 위반으로 INSERT 가 실패한다. 원인을 찾기 어려운 에러라 기억해둘 만하다.

@SpringBootApplication — 1곳

세 가지를 한꺼번에 켜다.

@SpringBootApplication	설정 클래스
@EnableAutoConfiguration	클래스패스를 보고 자동 설정
@ComponentScan	이 패키지 하위 전체를 스캔

이 클래스는 반드시 최상위 패키지에 있어야 한다. `com.yudillo.api` 에 있어서 `com.yudillo.api.auth`, `.user`, `.global` 을 전부 찾는다. 하위 패키지로 옮기면 나머지를 못 찾아 기동이 실패한다.

클래스 이름은 자유다. Gradle 플러그인이 `public static void main` 을 가진 클래스를 스캔해서 찾는다. `YudilloApiApplication` → `ApiApplication` 으로 바뀌어도 아무 문제가 없었던 이유다.

3. 웹 — 요청 매핑과 응답

@RestController — 2곳

@Controller + @ResponseBody

메서드 반환값을 뷰 이름이 아니라 응답 본문으로 취급한다. 객체를 반환하면 Jackson 이 JSON 으로 직렬화한다.

@Controller 만 쓰면 반환한 String 을 JSP/Thymeleaf 템플릿 이름으로 해석한다. REST API 에서는 항상 @RestController 다.

@RequestMapping — 2곳 (클래스 레벨)

```
@RestController
@RequestMapping("/api/auth")    // 공통 접두어
public class AuthController {

    @PostMapping("/login")       // → POST /api/auth/login
    public ApiResponse<TokenResponse> login(...) { ... }
}
```

HTTP 메서드별 매핑

애노테이션	이 프로젝트 사용처
<code>@GetMapping</code>	<code>/api/users/me</code>
<code>@PostMapping</code>	signup, login, reissue, logout
<code>@PatchMapping</code>	<code>/me/name</code> , <code>/me/password</code>
<code>@PutMapping</code>	현재 없음
<code>@DeleteMapping</code>	현재 없음

PATCH 와 PUT 의 차이

- `PUT` — 리소스 **전체** 교체. 보내지 않은 필드는 비워야 함
- `PATCH` — **일부만** 변경

이름만 바꾸는 것은 부분 변경이라 `PATCH` 가 맞다.

`@RequestBody` — 5곳

요청 본문(JSON)을 객체로 변환한다.

```
public ApiResponse<TokenResponse> login(@Valid @RequestBody LoginRequest request)
```

Jackson 이 JSON → `LoginRequest` record 로 역직렬화한다. **record** 는 생성자 바인딩이라 setter 가 없어도 된다.

`@ResponseStatus` — 1곳

성공 시 HTTP 상태코드를 지정한다.

```
@PostMapping("/signup")
@ResponseStatus(HttpStatus.CREATED)    // 201
public ApiResponse<UserResponse> signup(...) { ... }
```

`ResponseEntity` 대신 이걸 쓴 이유: `ResponseEntity.status(CREATED)` 는 런타임에 정해지는 값이라 **springdoc** 이 문서에 반영하지 못한다. 실제로는 201 인데 Swagger 에는 200 으로 나오던 문제가 있었다. 애노테이션으로 바꾸니 문서와 실재가 일치하고 코드도 짧아졌다.

`@RestControllerAdvice` — 1곳

```
@ControllerAdvice + @ResponseBody
```

모든 컨트롤러의 예외를 한곳에서 받는다. 컨트롤러에 `try-catch` 가 한 줄도 없는 이유다.

@ExceptionHandler — 5곳

어떤 예외를 처리할지 지정한다.

```
@ExceptionHandler(BusinessException.class)
public ResponseEntity<ErrorResponse> handleBusinessException(BusinessException e) { ... }
```

구체적인 타입이 우선한다. `BusinessException` 과 `Exception` 핸들러가 둘 다 있으면, `BusinessException` 이 발생했을 때 전자가 잡는다.

맨 아래 `Exception` 핸들러는 **보안 장치**다. 없으면 예상 못 한 예외에서 스택트레이스가 노출될 수 있다.

4. JPA — 엔티티 매핑

@Entity + @Table — 2곳

```
@Entity
@Table(name = "users")           // 테이블명 지정
public class User extends BaseEntity { ... }
```

`@Table` 을 생략하면 클래스명이 테이블명이 된다. `users` 로 지정한 이유는 `user` 가 PostgreSQL 예약어이기 때문이다. 예약어를 테이블명으로 쓰면 쿼리마다 따옴표가 필요해진다.

@Id + @GeneratedValue — 2곳

```
@Id
@GeneratedValue(strategy = GenerationType.UUID)    // User, 애플리케이션이 생성
private UUID id;

@Id
@GeneratedValue(strategy = GenerationType.IDENTITY) // RefreshToken, DB 가 생성
private Long id;
```

전략	동작	이 프로젝트
UUID	Hibernate 가 UUID 생성	User — URL 에 노출됨
IDENTITY	DB 의 auto_increment / bigserial	RefreshToken — 내부 식별자

기준: 밖으로 나가는 식별자는 **UUID**, 내부용은 **Long**. 순차 정수가 URL 에 노출되면 추측이 가능하고, `/api/users/1000` 같은 응답으로 규모까지 짐작된다.

@Column — 12곳

```
@Column(name = "email", nullable = false, unique = true, length = 255)
private String email;

@Column(name = "content", columnDefinition = "text")    // 타입 직접 지정
private String content;
```

속성	의미
<code>name</code>	컬럼명
<code>nullable = false</code>	NOT NULL
<code>unique = true</code>	UNIQUE 제약
<code>length</code>	varchar 길이
<code>updatable = false</code>	UPDATE 문에서 제외
<code>columnDefinition</code>	DDL 타입을 직접 지정

주의: `ddl-auto: validate` 라 이 값들이 DDL 을 만들지는 않는다. **Flyway** 가 만든 스키마와 대조하는 기준으로 쓰인다. 어긋나면 기동이 실패한다.

`updatable = false` 는 `BaseTimeEntity.createdAt` 에 쓴다. 생성 시각이 실수로 바뀌는 것을 물리적으로 막는다.

@Enumerated(EnumType.STRING) — 1곳

```
@Enumerated(EnumType.STRING)
@Column(name = "role", nullable = false, length = 20)
private Role role;
```

STRING 이 필수다. 기본값인 **ORDINAL** 은 순서(0, 1, 2)를 저장하는데, enum 상수를 중간에 추가하면 **기존 데이터의 의미가 전부 바뀐다**.

```
public enum Role { USER, ADMIN }           // ADMIN = 1
public enum Role { USER, MANAGER, ADMIN }   // MANAGER = 1 → 기존 ADMIN 이 MANAGER 로 해석됨
```

STRING 은 `"USER"`, `"ADMIN"` 을 저장하므로 순서와 무관하고, DB 를 직접 볼 때 읽기도 쉽다.

@MappedSuperclass + @EntityListeners — BaseTimeEntity

```
@Getter
@MappedSuperclass // 이 클래스는 테이블이 아님
@EntityListeners(AuditingEntityListener.class) // JPA 이벤트 리스너 등록
public abstract class BaseTimeEntity {

    @CreatedDate
    @Column(name = "created_at", nullable = false, updatable = false)
    private Instant createdAt;

    @LastModifiedDate
    @Column(name = "updated_at", nullable = false)
    private Instant updatedAt;
}
```

- @MappedSuperclass — 필드 매핑만 자식에게 물려준다. base_time_entity 테이블은 생기지 않는다
- @CreatedDate — INSERT 시 한 번만
- @LastModifiedDate — INSERT 와 UPDATE 마다

ApiApplication 의 @EnableJpaAuditing 이 있어야 동작한다.

Instant 를 쓴 이유: LocalDateTime 은 시간대 정보가 없어 서버 시간대가 바뀌면 같은 값이 다른 시각을 의미하게 된다. Instant 는 UTC 기준 절대 시각이라 모호함이 없고, Postgres 의 timestampz 와 짝이 맞는다.

@Index — 1곳

```
@Table(name = "refresh_tokens",
        indexes = @Index(name = "idx_refresh_tokens_user_id", columnList = "user_id"))
```

문서화 목적이 크다. 실제 인덱스는 Flyway SQL 이 만든다.

Spring Data JPA — 쿼리 메서드

애노테이션이 아니라 메서드 이름 규칙이다.

```
public interface UserRepository extends JpaRepository<User, UUID> {
    Optional<User> findByEmail(String email); // WHERE email = ?
    boolean existsByEmail(String email); // SELECT count(*) > 0 WHERE email = ?
}
```

구현 클래스가 없다. Spring Data 가 기동 시점에 프록시로 만든다. 이름에 오타가 있으면 기동 시점에 실패해서 바로 알 수 있다.

접두어	생성되는 쿼리
<code>findBy</code>	SELECT
<code>existsBy</code>	존재 여부 (count)
<code>countBy</code>	개수
<code>deleteBy</code>	DELETE

`existsBy` 를 쓰는 이유: 존재 확인만 필요할 때 엔티티 전체를 가져오지 않는다.

`findByEmail(email).isPresent()` 는 모든 컬럼을 SELECT 한다.

5. Bean Validation — 입력 검증

`@Valid` — 6곳

컨트롤러 파라미터에 붙여 검증을 **발동**시킨다.

```
public ApiResponse<TokenResponse> login(@Valid @RequestBody LoginRequest request)
```

이게 없으면 DTO 에 아무리 애노테이션을 붙여도 검증이 돌지 않는다. 실패하면

`MethodArgumentNotValidException` 이 발생하고 `GlobalExceptionHandler` 가 400 으로 변환한다.

`@NotBlank` — 10곳

```
@NotBlank(message = "이메일을 입력하세요.")
String email
```

null, 빈 문자열, 공백만 있는 문자열을 전부 거부한다.

애노테이션	<code>null</code>	<code>""</code>	<code>" "</code>
<code>@NotNull</code>	X	✓	✓
<code>@NotEmpty</code>	X	X	✓
<code>@NotBlank</code>	X	X	X

문자열에는 `@NotBlank` 가 거의 항상 맞다.

@Email — 2곳

이메일 형식을 확인한다. 다만 **RFC 기준이 느슨해서** `a@b` 같은 것도 통과한다. 엄격하게 하려면 `@Pattern` 을 함께 쓴다.

@Pattern — 4곳

정규식 검증이다. 비밀번호 규칙을 해부하면:

```
@Pattern(  
    regexp = "^(?=.*[A-Za-z])(?=.*\\d)(?=.*[!@#$%^&*~`-_=+{}|;:~'\">
    message = "형식에 맞는 비밀번호를 입력하세요.")
```

<code>^</code>	시작
<code>(?=.*[A-Za-z])</code>	전방탐색: 영문자가 최소 하나
<code>(?=.*\\d)</code>	전방탐색: 숫자가 최소 하나
<code>(?=.*[!@#\$%^&*~`-_=+{} ;:~'\"></code>	전방탐색: 특수문자가 최소 하나
<code>.{8,20}</code>	전체 8~20자
<code>\$</code>	끝

`(?=.*...)` 는 **전방탐색(lookahead)** 이다. 위치를 이동하지 않고 "뒤에 이런 게 있나"만 확인한다. 그래서 세 조건을 **순서 상관없이** 동시에 요구할 수 있다. `[A-Za-z].*\\d.*` 처럼 쓰면 "영문 다음에 숫자"라는 순서까지 강제된다.

이름 규칙은 더 단순하다.

```
@Pattern(regexp = "^[가-힣a-zA-Z0-9]{2,10}$")
```

한글·영문·숫자만 2~10자. 특수문자와 공백을 막아 표시 문제와 스푸핑을 줄인다.

@Size — 1곳

```
@Size(max = 255, message = "이메일이 너무 깁니다.")
```

문자열 길이나 컬렉션 크기를 제한한다.

검증 애노테이션은 문서에도 반영된다

springdoc 이 이것들을 읽어 OpenAPI 스펙에 넣는다. 따로 적을 필요가 없다.

검증	문서
@NotBlank	required: true (Swagger 에서 빨간 *)
@Size(max = 20)	maxLength: 20
@Pattern	pattern: "..."
@Email	format: email

6. Swagger (springdoc-openapi) — API 문서

@Tag — 2곳

Swagger UI 좌측의 그룹 이름이다.

```
@Tag(name = "Auth", description = "회원가입 / 로그인 / 토큰")
@RestController
public class AuthController { ... }
```

@Operation — 7곳

엔드포인트의 제목과 설명이다.

```
@Operation(summary = "로그인", description = "Access / Refresh 토큰을 발급한다.")
```

- `summary` — 엔드포인트 목록에 한 줄로 표시
- `description` — 펼쳤을 때 상세 설명

@Schema — 45곳 (가장 많이 쓰는 것)

클래스와 필드 양쪽에 붙는다.

```
@Schema(name = "SignupRequest", description = "회원가입 요청")    // 클래스
public record SignupRequest(

    @Schema(description = "이메일", example = "user@example.com")  // 필드
    String email,
    ...
) { }
```

속성	용도
<code>name</code>	스키마 이름 (Schemas 섹션)
<code>description</code>	설명
<code>example</code>	Example Value 에 표시될 값
<code>hidden</code>	문서에서 숨김

함정 — 제네릭 클래스에 `name` 을 고정하면 안 된다

실제로 겪은 문제다.

```
// 문제 있던 코드
@Schema(name = "ApiResponse", description = "공통 성공 응답")
public record ApiResponse<T>(boolean success, T data, String message) { }
```

`name` 을 고정하면 springdoc 이 `ApiResponse<UserResponse>` 와 `ApiResponse<TokenResponse>` 를 같은 스키마 하나로 취급한다. 제네릭 정보가 날아가 Swagger 에서 `data` 가 빈 객체로 나온다.

`name` 을 지우면 제네릭별로 만들어진다.

```
ApiResponseUserResponse
ApiResponseTokenResponse
ApiResponseVoid
```

제네릭 래퍼 클래스에는 `name` 을 쓰지 않는다.

`example` 은 수동, 나머지는 자동

```
@Schema(example = "홍길동") String name // 내가 적은 값
Role role // 없으면 → enum 값 목록 자동 표시
UUID id // 없으면 → "string" 으로 표시
```

`example` 은 정적 문서용 상수다. 실제 DB 값이나 API 호출 결과와 무관하다. Swagger UI 는 Example Value 를 그릴 때 서버에 요청을 보내지 않는다.

반면 타입·필수 여부·길이 제한·enum 값은 코드에서 자동으로 추출된다.

`@Parameter(hidden = true)` — 1곳

파라미터를 문서에서 숨긴다. `@CurrentUser` 안에 들어 있다.

```
@AuthenticationPrincipal
@Parameter(hidden = true)
public @interface CurrentUser { }
```

숨기지 않으면 Swagger 화면에 "authUser 를 입력하세요" 입력칸이 생긴다. 실제로는 토큰에서 자동으로 채워지는 값이라 사용자가 넣을 게 아니다.

@SecurityRequirements — 4곳

값이 없는 복수형 애노테이션이다. "이 엔드포인트는 인증이 필요 없다"를 뜻한다.

```
@SecurityRequirements          // ← 값 없음
@PostMapping("/signup")
```

SwaggerConfig 에서 전역으로 bearerAuth 를 요구하도록 선언했기 때문에, 공개 엔드포인트는 이걸로 해제해야 한다.

```
// SwaggerConfig
return new OpenAPI()
    .components(new Components().addSecuritySchemes("bearerAuth", bearerScheme))
    .addSecurityItem(new SecurityRequirement().addList("bearerAuth")); // 전역 요구
```

@SecurityRequirement (단수, 값 있음)와 헷갈리기 쉽다. 단수는 "이 스킴이 필요하다", 복수+빈 값은 "아무것도 필요 없다"이다.

이 프로젝트에서는 부수 효과도 있다. ErrorResponseCustomizer 가 이 값으로 인증 필요 여부를 판별해 401 응답을 자동으로 붙인다.

```
private boolean requiresAuth(Operation operation) {
    // springdoc 은 @SecurityRequirements 가 붙으면 security 를 빈 목록으로 설정한다
    return operation.getSecurity() == null || !operation.getSecurity().isEmpty();
}
```

SecurityConfig 의 PUBLIC_ENDPOINTS 와 짝을 맞춰야 한다. 한쪽만 고치면 문서와 실제 동작이 어긋난다.

7. Spring Security

@EnableMethodSecurity — 1곳

@PreAuthorize 같은 메서드 단위 권한 검사를 켜다.

```
@Configuration
@EnableMethodSecurity
public class SecurityConfig { ... }
```

현재 @PreAuthorize 를 쓰는 곳은 없다. 관리자 기능이 생기면 바로 쓸 수 있게 커뤘다.

@AuthenticationPrincipal — 1곳 (@CurrentUser 안)

SecurityContext 의 인증 주체를 컨트롤러 파라미터에 주입한다.

```
public ApiResponse<UserResponse> getMe(@CurrentUser AuthUser authUser)
//                                     ↳ @AuthenticationPrincipal 이 안에 들어있음
```

SecurityContextHolder 를 직접 쓰는 것보다 나은 점

```
// 이렇게도 가능하지만
Authentication auth = SecurityContextHolder.getContext().getAuthentication();
AuthUser user = (AuthUser) auth.getPrincipal();    // 형변환 실패 가능
```

1. 형변환이 없다
2. 메서드 시그니처만 봐도 "이 API 는 로그인이 필요하다"가 보인다
3. 테스트에서 인수로 넘기면 된다 (SecurityContext 세팅 불필요)

8. Jackson — JSON 직렬화

@JsonInclude(JsonInclude.Include.NON_NULL) — 3곳

null 필드를 JSON 에서 제외한다.

```
@JsonInclude(JsonInclude.Include.NON_NULL)
public record ApiResponse<T>(boolean success, T data, String message) { }
```

```
// message 가 null 일 때
{"success": true, "data": {...}}           // "message": null 이 안 나감

// data 가 null 일 때 (로그아웃)
{"success": true, "message": "로그아웃되었습니다."}
```

ApiResponse , ErrorResponse 에 붙어 있다. errors 배열이 검증 실패일 때만 나오는 것도 이 덕분이다.

@JsonFormat — 현재 0곳 (주석으로만 언급)

원래 ErrorResponse.timestamp 에 붙였다가 전역 설정으로 옮기면서 제거했다.

```
// 제거한 코드
@JsonFormat(shape = JsonFormat.Shape.STRING,
            pattern = "yyyy-MM-dd HH:mm:ss",
            timezone = "Asia/Seoul")
Instant timestamp
```

DTO 마다 붙이면 빠뜨리는 필드가 생기고 형식이 제각각이 된다. 지금은 `JacksonConfig` 가 모든 `Instant` 를 일괄 처리한다.

다만 알아둘 것: 개별 필드에 `@JsonFormat` 을 붙이면 **그 필드만 전역 설정을 무시한다**(필드 우선). 특정 필드만 다르게 해야 할 때 쓸 수 있다.

`timezone` 이 필수라는 점도 기억할 만하다. `Instant` 는 시간대가 없어서 패턴만으로는 포맷할 수 없고, 빼면 이런 예외가 난다.

```
UnsupportedTemporalTypeException: Unsupported field: YearOfEra
```

Jackson 3 주의 (Spring Boot 4)

Boot 4 는 **Jackson 3** 을 쓴다. 패키지가 바뀌었다.

	Jackson 2	Jackson 3
그룹	<code>com.fasterxml.jackson.core</code>	<code>tools.jackson.core</code>
스프링이 등록하는 빈	<code>ObjectMapper</code>	<code>JsonMapper</code> (<code>tools.jackson.databind.json</code>)
직렬화기	<code>JsonSerializer</code>	<code>ValueSerializer</code>
컨텍스트	<code>SerializerProvider</code>	<code>SerializationContext</code>
애노테이션	<code>com.fasterxml.jackson.annotation</code>	그대로

애노테이션만 예전 패키지를 그대로 쓴다는 게 헛갈리는 지점이다. `@JsonInclude` 의 import 가 `com.fasterxml...` 인 이유다.

```
import com.fasterxml.jackson.annotation.JsonInclude; // 애노테이션은 그대로
import tools.jackson.databind.json.JsonMapper;      // 빈은 새 패키지
```

`ObjectMapper` 를 주입하면 기동이 실패한다.

```
No qualifying bean of type 'com.fasterxml.jackson.databind.ObjectMapper' available
```

9. 직접 만든 애노테이션 3개

자바에서 애노테이션을 정의하는 문법과, 이 프로젝트가 그것을 쓰는 방식이다.

애노테이션 정의 문법

```
@Documented                // Javadoc 에 표시
@Target(ElementType.METHOD) // 붙일 수 있는 위치
@Retention(RetentionPolicy.RUNTIME) // 언제까지 유지
public @interface ApiErrorCodes {
    ErrorCode[] value();      // 속성
}
```

메타 애노테이션	의미
<code>@Target</code>	<code>METHOD</code> , <code>PARAMETER</code> , <code>TYPE</code> , <code>FIELD</code> 등
<code>@Retention</code>	<code>SOURCE</code> (컴파일 후 버림) / <code>CLASS</code> (기본) / <code>RUNTIME</code> (리플렉션 가능)
<code>@Documented</code>	Javadoc 포함

`@Retention(RUNTIME)` 이 필수다. 스프링과 springdoc 이 런타임에 리플렉션으로 읽기 때문에, 기본값(`CLASS`) 이면 아예 보이지 않는다. 조용히 동작하지 않아 원인을 찾기 어렵다.

`value()` 라는 이름의 속성은 특별하다. 속성이 하나뿐이면 이름을 생략할 수 있다.

```
@ApiErrorCodes(ErrorCode.INVALID_INPUT)      // value = 생략
@ApiErrorCodes(value = {ErrorCode.INVALID_INPUT, ...}) // 같은 뜻
```

배열 속성은 원소가 하나면 중괄호도 생략된다.

`@CurrentUser` — 메타 애노테이션 묶기

```
@Documented
@Target(ElementType.PARAMETER)
@Retention(RetentionPolicy.RUNTIME)
@AuthenticationPrincipal      // ← 다른 애노테이션을 붙인다
@Parameter(hidden = true)     // ← 이것도
public @interface CurrentUser { }
```

애노테이션 정의에 다른 애노테이션을 붙이면 **그것들이 함께 적용**된다. 반복을 줄이는 기법이다.

```
// 이렇게 쓰지 않아도 됨
public ApiResponse<UserResponse> getMe(
    @Parameter(hidden = true) @AuthenticationPrincipal AuthUser authUser) { ... }

// 이렇게 씀
public ApiResponse<UserResponse> getMe(@CurrentUser AuthUser authUser) { ... }
```

```
@ApiErrorCodes({ErrorCode.INVALID_INPUT, ErrorCode.DUPLICATE_EMAIL})
@ApiResponseMessage(SuccessMessage.SIGNUP)
@PostMapping("/signup")
```

Swagger 애노테이션을 직접 쓰지 않는 이유가 두 가지 있다.

1. 우리 `ApiResponse` 클래스와 Swagger 의 `@ApiResponse` 애노테이션이 이름 충돌한다. 같은 파일에서 둘 다 쓰려면 FQN 을 써야 한다.
2. 상태코드·메시지를 손으로 적으면 `ErrorCode` 와 어긋난다.

enum 만 나열하면 커스터마이저가 상태코드·설명·예시 JSON 을 전부 생성한다. 메시지를 고치면 문서가 자동으로 따라 바뀐다.

애노테이션을 읽는 쪽 — 리플렉션

```
// ErrorResponseCustomizer
@ApiErrorCodes declared = handlerMethod.getMethodAnnotation(ApiErrorCodes.class);
if (declared != null) {
    codes.addAll(List.of(declared.value()));
}
```

`HandlerMethod` 는 springdoc 이 넘겨주는 컨트롤러 메서드 정보다. 여기서 애노테이션을 꺼내 쓴다.

10. 자바 문법

record — 14곳

자바 16 에서 정식 도입된 불변 데이터 클래스다.

```
public record LoginRequest(String email, String password) { }
```

이 한 줄이 만드는 것:

- `private final` 필드 2개
- 모든 필드를 받는 생성자
- `email()`, `password()` 접근자 — `get` 접두어가 없다
- `equals()`, `hashCode()`, `toString()`

compact 생성자 — 기본값과 검증

```
public record DateTimeProperties(String pattern, String zone) {

    public DateTimeProperties {                // 파라미터 목록이 없다
        pattern = (pattern == null || pattern.isBlank()) ? "yyyy-MM-dd HH:mm:ss" : pattern.trim();
        zone     = (zone == null || zone.isBlank()) ? "Asia/Seoul" : zone.trim();
    }
}
```

`public DateTimeProperties {` — 괄호와 파라미터가 없는 형태다. 파라미터를 가공한 뒤 자동으로 필드에 대입된다. 기본값 처리나 검증에 쓴다.

`JwtProperties`, `CorsProperties` 도 같은 방식이다.

static 팩토리 메서드

```
public record UserResponse(UUID id, String email, ...) {

    public static UserResponse from(User user) {                // 인수 1개 → from
        return new UserResponse(user.getId(), user.getEmail(), ...);
    }
}

public record TokenResponse(String accessToken, ...) {

    public static TokenResponse of(String accessToken, String refreshToken, long expiresIn) {
        return new TokenResponse(accessToken, refreshToken, "Bearer", expiresIn); // 인수 2개+ → of
    }
}
```

`from` / `of` 는 자바 관례다. 변환 로직을 DTO 안에 두면 "이 API 가 무엇을 노출하는지"를 한 파일에서 볼 수 있다.

중첩 record

```
public record ErrorResponse(..., List<FieldError> errors, Instant timestamp) {

    public record FieldError(String field, String reason) { }    // 안에 정의
}
```

`ErrorResponse.FieldError` 로 참조한다. 이 타입이 밖에서 단독으로 쓰일 일이 없으므로 안에 두는 게 맞다.

이 중첩 record 때문에 Swagger 에서 `$ref` 가 깨진 적이 있다. `ModelConverters.read()` 는 최상위 스키마만 등록해서 `FieldError` 가 빠졌다. `readAll()` 로 고쳤다.

record 를 못 쓰는 곳 — 엔티티

JPA 는 기본 생성자와 필드 변경을 요구한다. record 는 불변이라 맞지 않는다. 그래서 엔티티는 일반 클래스로 두고 setter 를 막는 방식으로 불변성을 흉내낸다.

단순 상수가 아니라 필드와 메서드를 가진 enum 을 쓴다.

```
@Getter
public enum ErrorCode {

    INVALID_INPUT(HttpStatus.BAD_REQUEST, "C001", "입력값이 올바르지 않습니다."),
    DUPLICATE_EMAIL(HttpStatus.CONFLICT, "U002", "이미 사용 중인 이메일입니다.");

    private final HttpStatus status;
    private final String code;
    private final String message;

    ErrorCode(HttpStatus status, String code, String message) { // 생성자는 항상 private
        this.status = status;
        this.code = code;
        this.message = message;
    }
}
```

enum 으로 만든 이유: 컴파일 시점에 존재가 보장된다. `ErrorCode.DUPLICATE_EMAIL` 처럼 오타를 내면 컴파일 이 실패한다. 문자열 상수로 관리하면 런타임까지 모른다.

메서드를 가진 enum

```
public enum Role {
    USER, ADMIN;

    public String authority() {
        return "ROLE_" + name(); // name() 은 enum 이 기본 제공
    }
}
```

Spring Security 의 `hasRole("ADMIN")` 은 내부적으로 `ROLE_ADMIN` 을 찾는다. 접두어를 안 붙이면 **영원히 false** 가 된다. 이 변환을 enum 안에 캡슐화했다.

`TokenType` (ACCESS/REFRESH), `SuccessMessage` 도 enum 이다.

람다와 메서드 참조

```
// 람다
.csrf(csrf -> csrf.disable())
.sessionManagement(session -> session.sessionCreationPolicy(STATELESS))

// 메서드 참조 (더 짧음)
.csrf(AbstractHttpConfigurer::disable)
.map(AuthUser::from)
```

`::` 는 **메서드 참조**다. `x -> x.method()` 를 `Type::method` 로 줄인 것이다.

형태	예	뜻
<code>Type::staticMethod</code>	<code>AuthUser::from</code>	정적 메서드
<code>Type::instanceMethod</code>	<code>ProjectMember::getRole</code>	인스턴스 메서드 (첫 인수가 수신자)
<code>object::method</code>	<code>this::customize</code>	특정 객체의 메서드

`SecurityConfig` 에서 람다를 쓰는 이유는 Spring Security 6+ 가 **람다 DSL 만 지원**하기 때문이다. 예전의 `.csrf().disable().and()` 체이닝 방식은 제거됐다.

Optional

"값이 없을 수 있다"를 타입으로 표현한다.

```
Optional<User> findByEmail(String email);

// 사용
User user = userRepository.findByEmail(email)
    .orElseThrow(() -> new BusinessException(ErrorCode.INVALID_CREDENTIALS));
```

null 을 반환하면 호출부가 null 체크를 빠뜨려 NPE 가 난다. `Optional` 은 `.orElseThrow()` 를 강제하므로 처리를 잊지 못한다.

```
// 체이닝
return userRepository.findById(userId)
    .map(AuthUser::from) // 있으면 변환
    .orElseThrow(() -> new JwtAuthenticationException(...)); // 없으면 예외
```

`boardListRepository.findMaxPosition()` 처럼 `Optional<Double>` 을 반환하는 경우도 있었다 — `max()` 가 행이 없으면 null 을 주기 때문이다.

Stream API

```
List<ErrorResponse.FieldError> errors = e.getBindingResult().getFieldErrors().stream()
    .map(fe -> new ErrorResponse.FieldError(fe.getField(), fe.getDefaultMessage()))
    .toList();
```

`.toList()` 는 자바 16+ 다. 예전의 `.collect(Collectors.toList())` 보다 짧고, **불변 리스트**를 반환한다.

`groupingBy` 로 묶는 예도 있다.

```
// ErrorResponseCustomizer - 같은 상태코드끼리 묶는다
return codes.stream()
    .distinct()
    .collect(Collectors.groupingBy(
        code -> String.valueOf(code.getStatus().value()),
        LinkedHashMap::new, // 순서 유지
        Collectors.toList()));
```

`LinkedHashMap::new` 를 준 이유는 **삽입 순서를 유지**하기 위해서다. 기본 `HashMap` 은 순서가 보장되지 않아 문서의 응답 순서가 매번 달라진다.

switch 식 — 1곳

자바 14+ 의 표현식 형태 switch 다.

```
return switch (String.valueOf(schema.getType())) {
    case "boolean" -> true;
    case "integer" -> 0;
    case "number"  -> 0.0;
    case "object"  -> Map.of();
    default        -> "string";
};
```

`break` 가 필요 없고, **값을 반환**한다. 예전 형태와 비교하면:

```
// 예전
Object result;
switch (type) {
    case "boolean": result = true; break;    // break 를 빠뜨리면 아래로 흘러감
    default: result = "string";
}
```

패턴 매칭 instanceof — 1곳

자바 16+ 다.

```
if (example instanceof Map<?, ?> map && map.containsKey("message")) {
    // map 을 바로 쓸 수 있다. 형변환 불필요
}
```

예전에는 이렇게 써야 했다.

```
if (example instanceof Map) {
    Map<?, ?> map = (Map<?, ?>) example;    // 형변환을 또 써야 함
    ...
}
```

제네릭

```
public record ApiResponse<T>(boolean success, T data, String message) {

    public static <T> ApiResponse<T> ok(T data) { ... }
    //      └ 메서드 레벨 타입 파라미터
}
```

`ApiResponse<UserResponse>` , `ApiResponse<TokenResponse>` , `ApiResponse<Void>` 로 쓰인다.

`Void` 는 "반환할 데이터가 없음"을 뜻한다. `data` 가 `null` 이 되고 `@JsonInclude(NON_NULL)` 로 JSON 에서 빠진다.

와일드카드도 쓴다.

```
private final Collection<? extends GrantedAuthority> authorities;
//                      ↳ GrantedAuthority 의 하위 타입이면 무엇이든
```

인터페이스 구현 — 4곳

```
public class AuthUser implements UserDetails { ... }
public class JwtAuthenticationFilter extends OncePerRequestFilter { ... }
public class AuthUserDetailsService implements UserDetailsService { ... }
public class JwtAuthenticationEntryPoint implements AuthenticationEntryPoint { ... }
```

전부 **프레임워크의 확장 지점**이다. 스프링이 정한 규약을 구현해 끼워 넣는 방식이다.

`@Override` 는 7곳에 쓰인다. **오타로 인한 버그를 막는다** — 시그니처가 틀리면 컴파일이 실패한다.

텍스트 블록 — 현재 미사용

자바 15+ 의 여러 줄 문자열이다. JPQL 을 쓸 때 유용했다.

```
// 프로젝트 도메인에 있던 코드 (현재는 삭제됨)
@Query("""
    select p
    from Project p
    join fetch p.owner
    where exists (select 1 from ProjectMember pm where pm.project = p)
    """)
```

칸반 도메인을 다시 만들면 쓰게 된다.

11. 헛갈리기 쉬운 것들

실제로 이 프로젝트에서 문제가 됐거나, 자주 틀리는 지점만 모았다.

@Transactional 이 안 먹는 경우

```
@Service
public class SomeService {
    @Transactional
    public void a() {
        b(); // ← 내부 호출. 프록시를 안 거쳐서 b() 의 설정이 무시된다
    }

    @Transactional(propagation = REQUIRES_NEW)
    public void b() { ... }
}
```

전파 옵션을 다르게 하려면 클래스를 분리해야 한다.

@Schema(name = ...) 을 제네릭에 쓰면 안 된다

`ApiResponse<T>` 에 `name` 을 고정하면 제네릭별 스키마가 하나로 뭉개진다.

@Retention 기본값은 CLASS

커스텀 애노테이션에 `@Retention(RUNTIME)` 을 안 붙이면 리플렉션으로 읽을 수 없다. **에러 없이 조용히 무시** 되므로 원인 찾기가 어렵다.

@Bean 메서드명과 @Configuration 클래스명이 겹치면 기동 실패

```
The bean 'errorResponseCustomizer' ... already been defined
```

EnumType.ORDINAL (기본값) 은 위험하다

반드시 `@Enumerated(EnumType.STRING)` 을 명시한다.

hasRole("ADMIN") 은 ROLE_ADMIN 을 찾는다

권한 문자열에 `ROLE_` 접두어를 직접 붙여야 한다.

@Valid 없이는 검증이 안 돈다

DTO 에 애노테이션을 아무리 붙여도 컨트롤러 파라미터에 `@Valid` 가 없으면 무시된다.

spring.jackson.date-format 은 java.time 에 안 먹는다

`java.util.Date`, `Calendar` 전용이다. `Instant`, `LocalDateTime` 에는 효과가 없다. 이 프로젝트가 커스텀 직렬화기를 만든 이유다.

BCrypt 는 해시끼리 비교하면 안 된다

```
passwordEncoder.encode(input).equals(storedHash) // 항상 false - salt 가 매번 다름
passwordEncoder.matches(input, storedHash) // 이게 맞다
```

ResponseBody 는 문서화되지 않는다

`ResponseBody.status(CREATED)` 는 런타임 값이라 springdoc 이 못 읽는다. `@ResponseStatus` 를 쓴다.

Boot 4 에서 ObjectMapper 를 주입하면 실패

`tools.jackson.databind.json.JsonMapper` 를 쓴다.

12. 한눈에 보는 애노테이션 지도

파일을 열었을 때 어떤 애노테이션을 보게 되는지 정리했다.

컨트롤러

```
@Tag                                ← Swagger 그룹
@RestController                     ← 스프링: JSON 반환 컨트롤러
@RequestMapping("/api/..")         ← 스프링: 공통 경로
@RequiredArgsConstructor             ← Lombok: 생성자 주입
public class XxxController {

    @ApiErrorCodes({...})           ← 직접 만들: 에러 문서화
    @ApiSuccessMessage(...)         ← 직접 만들: 성공 문구 문서화
    @Operation(summary = ...)       ← Swagger: 엔드포인트 설명
    @SecurityRequirements            ← Swagger: 인증 불필요 선언
    @PostMapping("/xxx")            ← 스프링: HTTP 매핑
    @ResponseStatus(CREATED)        ← 스프링: 성공 상태 코드
    public ApiResponse<T> xxx(@CurrentUser AuthUser user,           ← 직접 만들
                             @Valid @RequestBody XxxRequest r)     ← 검증 + 본문 바인딩
}
```

서비스

```
@Slf4j                                ← Lombok: 로거
@Service                              ← 스프링: 빈 등록
@RequiredArgsConstructor                ← Lombok: 생성자 주입
public class XxxService {

    @Transactional                    ← 쓰기
    @Transactional(readOnly = true)  ← 읽기

}
```

엔티티

```
@Getter                                ← Lombok
@Entity                                ← JPA
@Table(name = "users")                 ← JPA: 테이블명
@NoArgsConstructor(access = PROTECTED)  ← Lombok + JPA 요구사항
public class User extends BaseEntity {

    @Id                                ← JPA: PK
    @GeneratedValue(strategy = UUID)    ← JPA: 생성 전략
    @Column(name = "...", nullable = false) ← JPA: 컬럼 매핑
    @Enumerated(EnumType.STRING)        ← JPA: enum 저장 방식

    @Builder private User(...)          ← Lombok: 빌더

}
```

DTO

```
@Schema(name = "...", description = "...") ← Swagger: 모델 설명
public record XxxRequest(

    @Schema(description = "...", example = "...") ← Swagger: 필드 + 예시
    @NotBlank(message = "...")                  ← 검증
    @Pattern(regexp = "...", message = "...")    ← 검증
    String field

) { }
```

설정

```
@Configuration                                ← 스프링: 설정 클래스
@EnableMethodSecurity                          ← Security: 메서드 권한
@RequiredArgsConstructor                       ← Lombok
public class SecurityConfig {

    @Bean                                      ← 스프링: 빈 등록
    public SecurityFilterChain ...() { }
}

@Validated                                    ← 설정값 검증
@ConfigurationProperties(prefix = "..")        ← yml 바인딩
public record XxxProperties(...) { }
```

문서 끝. 2026-09-28 기준 코드에서 추출.